

Levitador Neumático

Fernando Bustamante

fernando.miguel.bustamante@gmail.com

Nahuel Portas

nahuluciano@gmail.com

Indice

1. Introducción.....	1	11. Referencias.....	6
2. ABSTRACT.....	1	ANEXO 1: DIAGRAMA DE	
3. Fundamentación.....	2	CONEXIONES.....	7
3.1. Problemática.....	2	ANEXO 2: DIAGRAMA DE TAREAS.....	8
3.2. Solución.....	2	ANEXO 3A: VISTA 3D DEL PCB (LADO	
3.4. Fundamentación de hardware.....	3	DE LOS COMPONENTES).....	9
3.5. Fundamentación de software.....	3	ANEXO 3B: VISTA 3D DEL PCB (LADO	
4. Objetivos.....	4	DEL COBRE).....	10
5. Alcance.....	4	ANEXO 4A: PCB PARA IMPRESO (LADO	
6. Diagrama en bloques.....	4	DEL COBRE) EN ESCALA 1:1.....	11
7. Especificaciones de hardware.....	4	ANEXO 4B: PCB PARA IMPRESO (LADO	
7.1. Plataforma Raspberry Pico.....	4	DE COMPONENTES) EN ESCALA 1:1.....	12
7.2. Sensor HC-SR04.....	4	ANEXO 5: ESQUEMÁTICO.....	13
7.3. Reloj de tiempo real (RTC).....	4	ANEXO 6: PLACA IMPRESA (LADO DEL	
7.4. Fuente de alimentación.....	4	COBRE).....	14
8. Especificaciones de software.....	5	ANEXO 6B: PLACA IMPRESA (LADO DE	
9. Esquemático Y PCB.....	5	LOS COMPONENTES).....	15
10. Conclusiones y mejoras.....	5		

1. Introducción

Este documento es una presentación del proyecto Flotador Neumático. Se pretenderá diseñar un sistema de control que permita sostener en el aire un elemento liviano como una pelota de telgopor. El sistemas de poder tener la posibilidad de regular a través de un potenciómetro o teclado la altura (*setpoint*) que se desea mantener y regular la intensidad del cooler para contenerlo.

Se proveerá una forma de almacenar la información del proyecto en forma de datalogger. La forma y los parámetros son a elección.

2. ABSTRACT

El proyecto consiste en un sistema controlado que mantendrá en flotación un un objeto esférico liviano cuyo material sera poliestireno expandido. Para el control se empleara un control PID, el cual sensara la altura a la cual se encuentra el objeto y la comparara con la altura seteada (T). Ademas de la altura seteada también se fijaran valores máximo (max) y mínimo (min). Todo esto se realizara sobre un sistema embebido como Raspberry Pico 2, utilizando su SDK C/C++ y programandole con un sistema operativo de tiempo real conocido como FreeRTOS.

PALABRAS CLAVES: Control PID, Raspberry Pico 2, FreeRTOS.

3. Fundamentación

Un levitador neumático es un dispositivo que utiliza aire u otro gas a presión para generar una fuerza que contrarresta la gravedad permitiendo que un objeto levite de manera estable en el aire sin contacto físico con superficies sólidas. Su funcionamiento de basa en el equilibrio entre:

- Flujo de aire a presión: El levitador expulsa aire a alta velocidad a través de una boquilla o superficie porosa.
- Fuerza de sustentación: El aire crea una zona de baja presión entre el objeto y el levitador (efecto Bernoulli o cojín de aire), generando una fuerza ascendente que contrarresta el peso del objeto.
- Estabilización: Algunos diseños usan sensores y sistemas de retroalimentación para ajustar el flujo de aire y mantener la levitación estable.

3.1. Problemática

Se presentan dos problemas para el proyecto.

La problemática físico-mecánica, en la cual se debe considerar el diseño y armado del sistema de levitación. En cuanto a esta parte debemos considera la fuente de flujo de aire, la hermeticidad del sistema físico-mecánico y el material del objeto, así como su peso y dimensiones. Para nuestro caso se usara una esfera de poliestireno expandido, cuyo peso a vencer por el flujo de aire no es demasiado grande, pero si el sistema no esta bien sellado para concentrar el flujo de aire se pueden generar perturbaciones las cuales representaran una carga para el sistema de control de lazo cerrado.

La problemática de control e instrumentación (hardware y software), en esta parte se presenta

el desafío de implementar un sistema de control a lazo cerrado con el objetivo de mantener la estabilidad del objeto en flotación. La incógnita a encontrar es como medir la distancia entre el objeto y la fuente de aire, así como la selección de un sensor para la medición de la altura, dicho sensor debe contar con un rango y error aceptable. Pero también se debe tener en cuenta el algoritmo en el cual debemos de convertir la señal del sensor en una acción de corrección, si bien el control se realizara con un PID se debe sintonizar las variables K_p , K_d y K_i .

3.2. Solución

Para la solución de la parte física-mecánica y sabiendo que el objeto es una esfera con peso y dimensiones conocidas, se propuso el uso de un tubo de acrílico transparente con un diámetro cercano al de la esfera con la intención de limitar los grados de libertad laterales, ademas en la parte inferior de la esfera se coloco una superficie plana la cual ayudara que todo el flujo de aire golpe la superficie.

La solución para la segunda parte de la problemática, es implementar el sistema de control a lazo cerrado (PID) sobre un microcontrolador, programado con un sistema operativo en tiempo real, para tal caso se uso FreeRTOS. El uso de un RTOS es la mejor opción ya que tenemos la ventaja de:

- Determinismo: permitiendo estructurar el programa en tareas bien definidas.
- Mantenebilidad: Separa las funcionalidades en módulos, permitiendo una mayor organización del código y futuras optimizaciones del código.
- Gestión de recursos: Se gestiona de manera eficiente el CPU, memoria y la comunicación ente las taras por medio de colas y/o semáforos.

La medición de la altura se realizara por medio de un sensor ultrasonido HC-SR04. Este sensor es digital por lo que no sera necesario una conversión analógica-digital, siendo su capacidad de medición ente 3cm y 400cm.

Como fuente de aire se utilizara un cooler controlado por PWM, en el mercado existen varios modelos cuyo PWM es de 3.3V, lo cual es ideal para la plataforma Pico 2. Ademas el PWM es la variable a la cual se le aplicara la corrección.

También se proveerá un medio para establecer la altura de flotación y la visualización de la altura medida y seteada. Se utilizara un LCD de 20x4, donde se vera: *setpoint* (T), *setpoint_max* (max), *setpoint_min* (min), medición obtenida del sensor.

3.4. Fundamentación de hardware

Para el armado de la parte física-mecánica, se selecciono una esfera de 9 gramos y 7.8 cm de diámetro y sus elementos serán:

Cooler: este elemento sera el que provee la fuente de aire, se selecciona un cooler ya que la fuerza de aire es suficiente para la elevar la esfera. Ademas se tiene la ventaja de un bajo costo y un consumo de energía mucho menor que el de un soplador.

Tubo de acrílico: tubo de aproximadamente 50 cm y un diámetro similar a la esfera. El diámetro no tiene que ser muy ajustado ya que la esfera podría rozar las paredes y afectar la respuesta del sistema. El tubo proporciona una guía física para el trayecto de la esfera.

Plancha de acrílico perforada: Esta plancha perforada permita tener un flujo de aire mas uniforme. La misma se colocara sobre el cooler y el tubo. Con la plancha se pretende un flujo laminar y uniforme.

Para el sistema de control se utiliza el sensor HC-SR04 que se encargara de medir la altura

entre la fuente de aire y la esfera, esta altura sera que se utilizara para la corrección.

La selección del *setpoint*, *setpoint_max* y *setpoint_min* se realizara mediante un potenciómetro por lo que se deberá usar un conversor ADC y el cambio entre las opciones se realizara por medio de un botón. La información *setpoint*, *setpoint_max*, *setpoint_min* y la altura se muestran en un LCD con conexión I2C.

Se utilizara un medio de almacenamiento como datalogger. La opción fue una memoria SD en la cual se almacenara un registro con fecha, hora, *setpoint*, *setpoint_max*, *setpoint_min*. Para la fecha y hora se usara un RTC que compartirá el mismo bus I2C que el LCD.

3.5. Fundamentación de software

El control de los elementos de hardware quedara a cargo de la plataforma Raspberry Pico 2 (RP2350) la cual sera programada usando FreeRTOS.

El uso de FreeRTOS permitirá dividir el código en tareas independientes de esta manera se asignaran prioridades a cada una de ellas. Se provechara la ventaja del determinismo lo cual permitirá que la adquisición de datos se realiza en tiempo real y sin perdida de información.

La adquisición de datos por parte del sensor no requiere de una conversión ADC por lo que solo nos limitaremos al muestro.

En cuanto al seteo de los datos utilizamos un potenciómetro para setear y un botón para cambiar entre las opciones. En el potenciómetro sera necesario realizar algunos ajusten ya que se debe convertir la magnitud eléctrica en una magnitud física.

$$H_{cm} = N \times \frac{3.3}{4095} \times F_a$$

La formula de arriba es la que nos permitirá pasar de la magnitud eléctrica a la física, donde F_a es el factor de adecuación.

4. Objetivos

En términos generales se pretende diseñar e implementar un levitador neumático en un sistema embebido que permita suspender de manera estable una esfera en un punto de referencia especificado.

- Establecer el valor de flotación (*setpoint*) de la esfera, además de valores máximo (*setpoint_max*) y mínimo (*setpoint_min*).
- Medir en todo momento el valor de la altura de la esfera.
- Corregir la altura para mantener el nivel al que flota la esfera.
- Adquisición de datos para llevar a cabo un registro de configuración.

5. Alcance

Se pretende mantener en suspensión la esfera con un error de ± 1 cm. Pero también proveer un medio de almacenamiento para los parámetros seteados. Adicionalmente se pretende almacenar los valores configurados junto a la fecha y hora para tener un registro en forma de datalogger.

6. Diagrama en bloques

En el ANEXO 1 se puede ver el diagrama de conexiones entre la plataforma Raspberry Pico 2 y los elementos de hardware, en el mismo también se indica que función cumple cada uno de los pines.

7. Especificaciones de hardware

Este apartado es para especificar las características de los principales módulos.

7.1. Plataforma Raspberry Pico

La plataforma Raspberry ofrece distintos modelos de placas de desarrollo, para el proyecto se utilizó el modelo Pico 2 (RP2350).

- Memoria flash 4MB
- Memoria RAM 512Kb
- Dos módulos SPI
- Dos módulos I2C
- Dos módulos UART
- 3 entradas ADC
- PWM: todos los pines pueden actuar como PWM
- Alimentación 5V por medio del pin VSYS

7.2. Sensor HC-SR04

Este sensor es uno de los más accesibles del mercado comparado con los infrarrojos, su bajo costo lo hace ideal para esta aplicación.

- Rango de operación 2cm a 400cm
- Angulo efectivo menor a 15°
- Resolución de 0.3cm
- Ancho de pulso para el disparo de 10 μ s
- Alimentación de 5V
- Corriente de trabajo 15mA

7.3. Reloj de tiempo real (RTC)

Reloj de tiempo real RTC DS3231.

- Testimonio de alimentación: 5V
- Cuenta: segundos, minutos, hora, día, mes, año
- Batería: tipo botón de 5V para mantener la configuración de la fecha

7.4. Fuente de alimentación

Todo el circuito será alimentado por una fuente switching de 12Vx2A. De la cual se extraen los 3.3V para el ADC y el módulo I2C, y los 5V para alimentar los sensores y otros módulos. Se

emplean dos reguladores LM7805 y LM317 ajustable a 3.3V.

8. Especificaciones de software

El Firmware cuenta con un total de 10 tareas, de las cuales 2 son para testeo del botón de opciones y 1 para testear el sensor de medición. La tarea **task_init** es la de mayor prioridad y solo configura los elementos de la plataforma y luego se elimina para recuperar recursos. El Diagrama de tareas se puede ver en el ANEXO 2.

El funcionamiento se puede resumir de la siguiente forma, la tarea **task_SetPoint** recibe la configuración del setpoint, al pulsar el botón pasara a la siguiente opción **setpoint_max**, al pulsar el boto se podra configurar el **setpoint_min**. El máximo y minimo esta limitado a un valor por defecto de setpoint+1 para el máximo y setpoint-1 para el minimo.

Por medio de colas se enviá la información de la configuración del seteo a las tareas **task_guardiana_lcd**, **task_pid**, **task_guardiana_sd**.

La tarea **task_guardiana_lcd** recibirá información de:

- **task_SetPoint**: Recibe la configuración de los setpoint.
- **task_pid**: Reibe la medico del sensor.

La tarea **task_guardiana_sd** recibirá información de:

- **task_SetPoint**: Recibe la configuración de los setpoint.
- **task_rtc**: Recibe información sobre la fecha y hora actual.

9. Esquemático Y PCB

Para la realización del esquemático y PCB se utilizo el software KiCAD versión 8.0. En el esquemático se divido el diseño en bloques:

- Bloque de sensores y modulo
- Bloque de alimentación
- Bloque de banderas

El esquemático se puede ver en el ANEXO 3. Se fijaran punto de prueba (test point), los cuales se pueden reconocer por la nomenclatura **TEST_XXX**. Los puntos de prueba disponibles son 4, dos para el sensor en los pines ECHO y TRIG, dos para el cooler donde mediremos el PWM y la velocidad del cooler (RPM.)

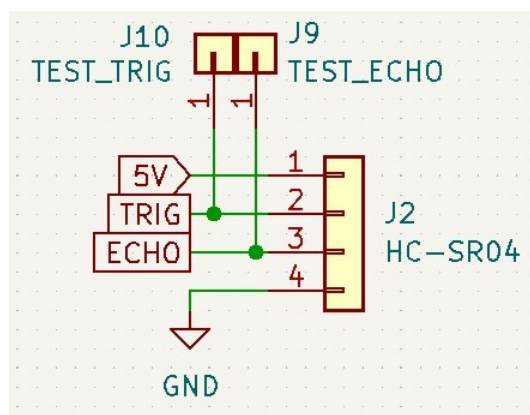


Figura 1. Testpoint para el HC-SR04

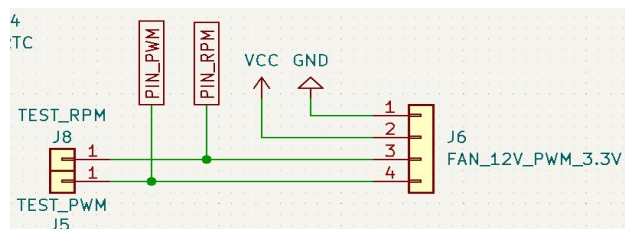


Figura 2. Testpoint para RPM y PWM del cooler

10. Conclusiones y mejoras

En base al diseño tanto de la parte físico-mecánica como electrónica podemos concluir lo siguiente:

- La construcción del elemento físico-mecánico debe ser hermética a fin de evitar fugas de aire.
- Se debe proveer una superficie plana para que el flujo de aire golpee de manera uniforme.

- Se debe colocar una plancha perforada a fin de tener un flujo laminar.
- Es importante poder lograr los primeros tres apartados, ya que esto reducirá el esfuerzo que debe realizar el controlador.
- La plataforma Raspberry con el SDK C/C++, permitió una programación más fluida en conjunto con el kernel de FreeRTOS. Principalmente por las APIs de FreeRTOS que son directas y muy intuitivas, y de las APIs de Raspberry que permiten una mayor abstracción de los elementos de hardware.
- El uso de semáforo mutex en la administración del bus I2C permitió una mayor fluidez de los datos y que estos no se choquen y los dispositivos actúen en tiempos necesarios para finalizar su transferencia de datos.
- El uso de las colas permitió transferir la información de manera segura y directa, sin la necesidad de variables globales que podrían ser manipuladas por terceras tareas o funciones, resultando en datos corruptos.

11. Referencias

[1] FreeRTOS:

<https://www.freertos.org/>

[2] Plataforma Raspberry:

<https://www.raspberrypi.com/documentation/microcontrollers/pico-series.html>

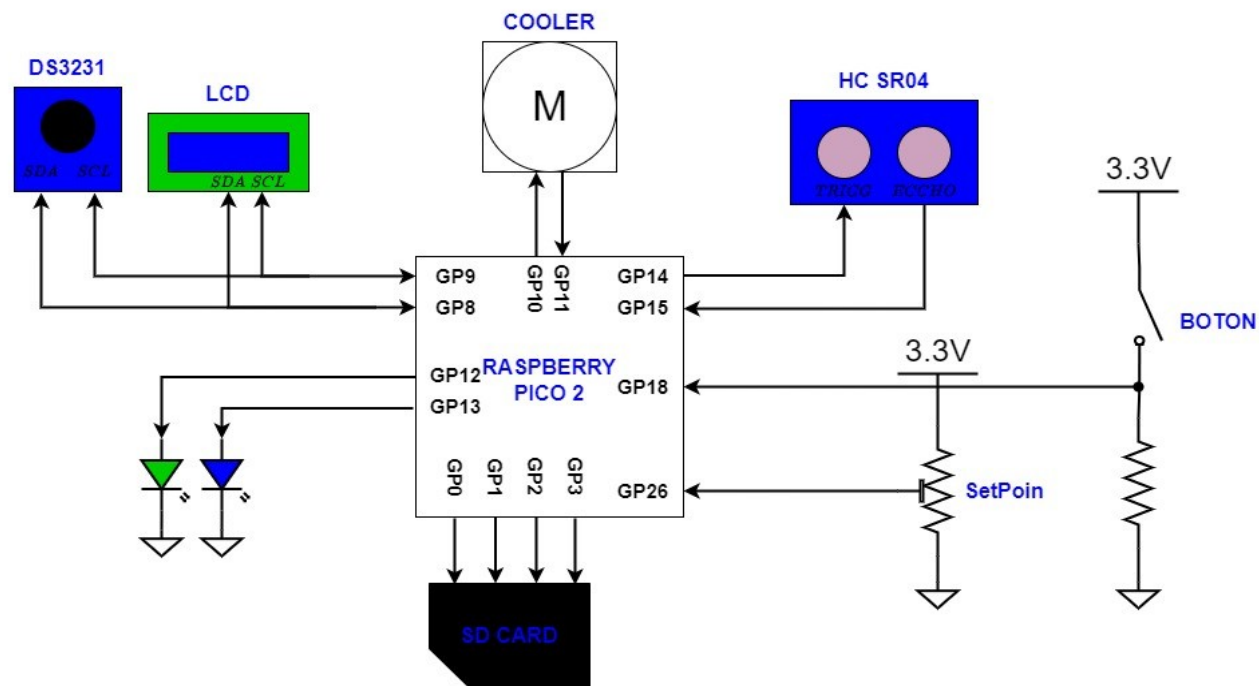
https://www.raspberrypi.com/documentation/microcontrollers/c_sdk.html

[3] Hojas de datos:

<https://www.alldatasheet.es/datasheet-pdf/pdf/1132203/ETC2/HC-SR04.html>

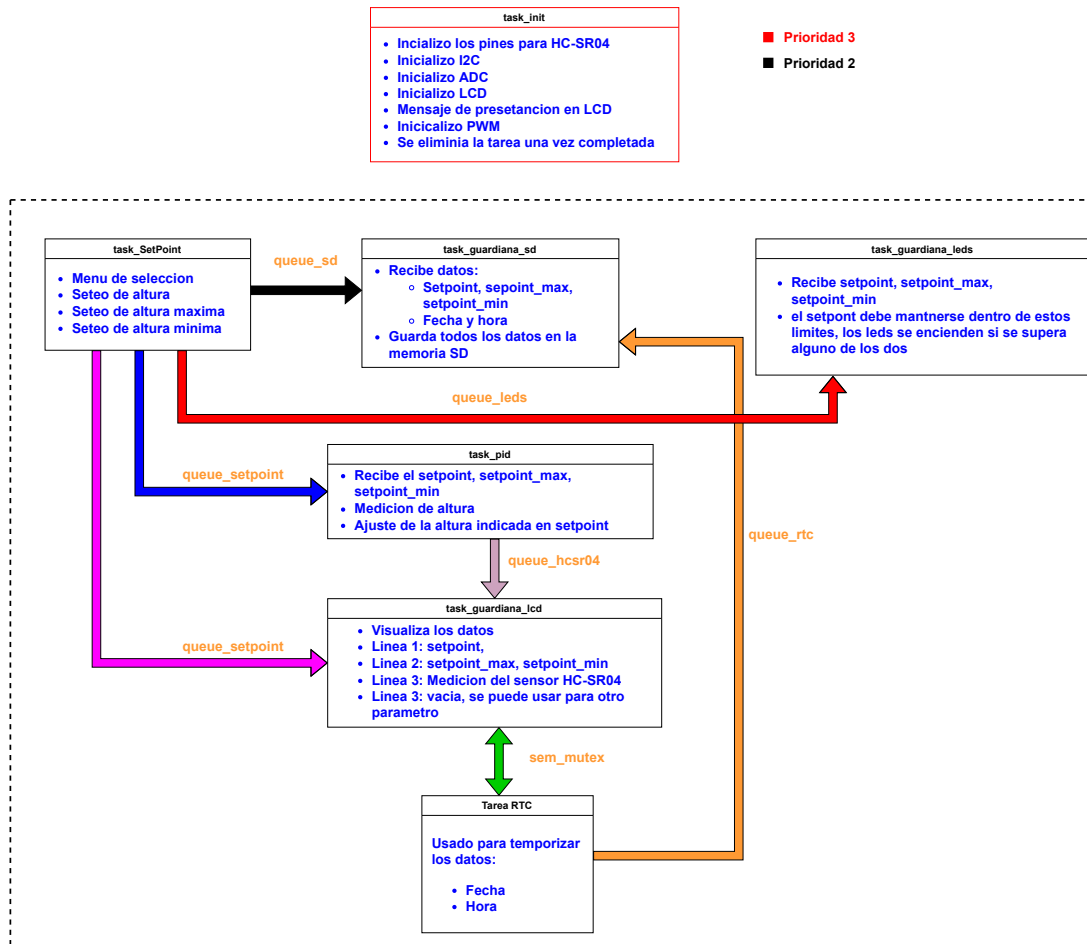
<https://support.arctic.de/p8max>

ANEXO 1: DIAGRAMA DE CONEXIONES

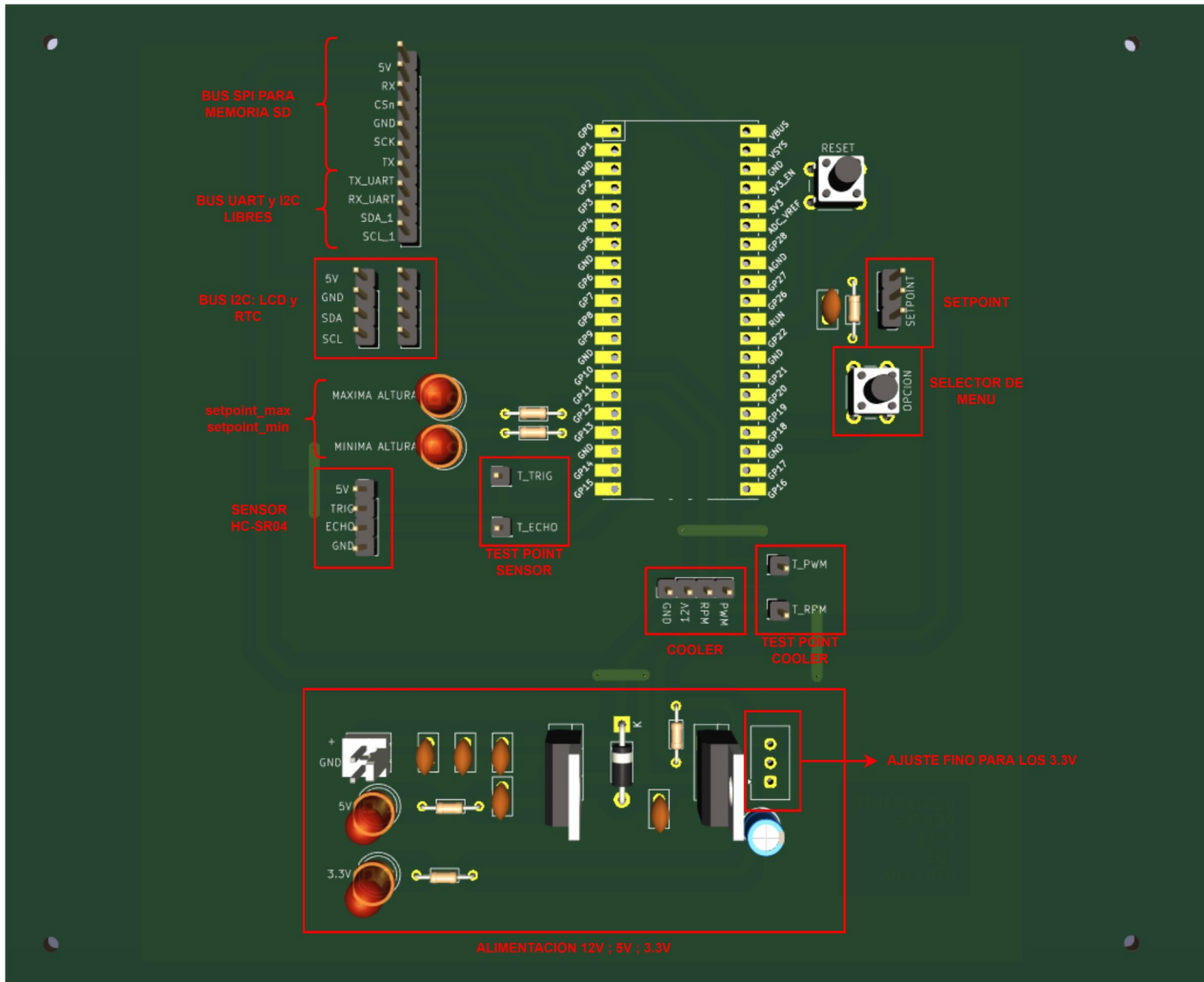


INFORMACION DE PINES	
GP8 ; GP9	SDA, SCL
GP15 ; GP16	TRIGG, ECHHO
GP10, GP11	PWM ; RPM
GP12, GP13	MAXIMO ; MINIMO
GP0, GP1, GP2, GP3	MISO, CSn, SCK, MOSI
GP18	Pin 24, BOTON

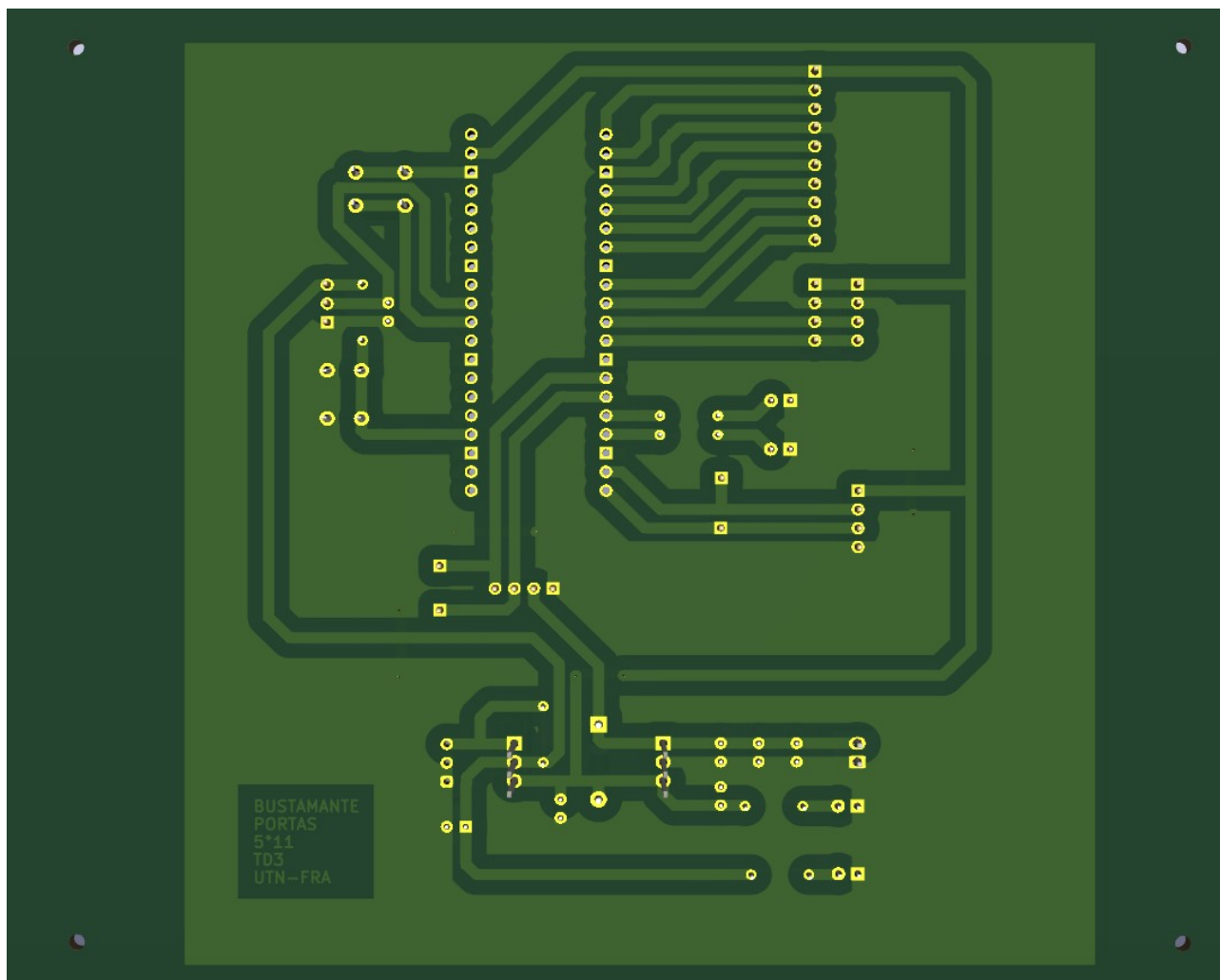
ANEXO 2: DIAGRAMA DE TAREAS



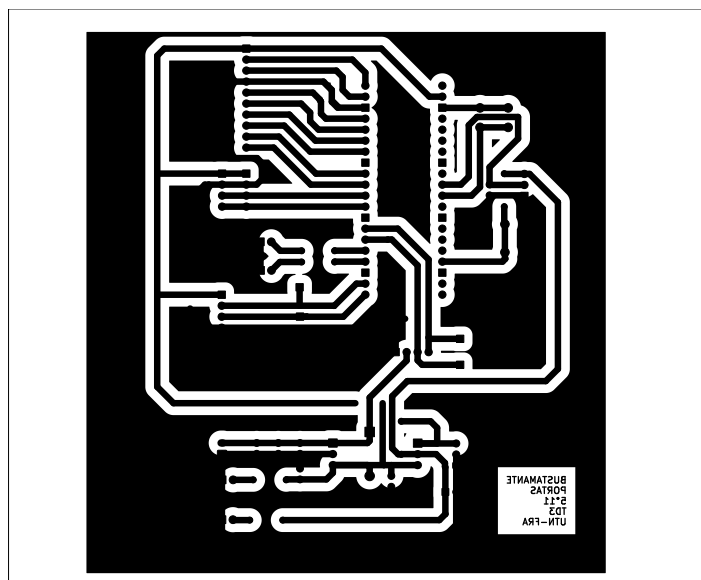
ANEXO 3A: VISTA 3D DEL PCB (LADO DE LOS COMPONENTES)



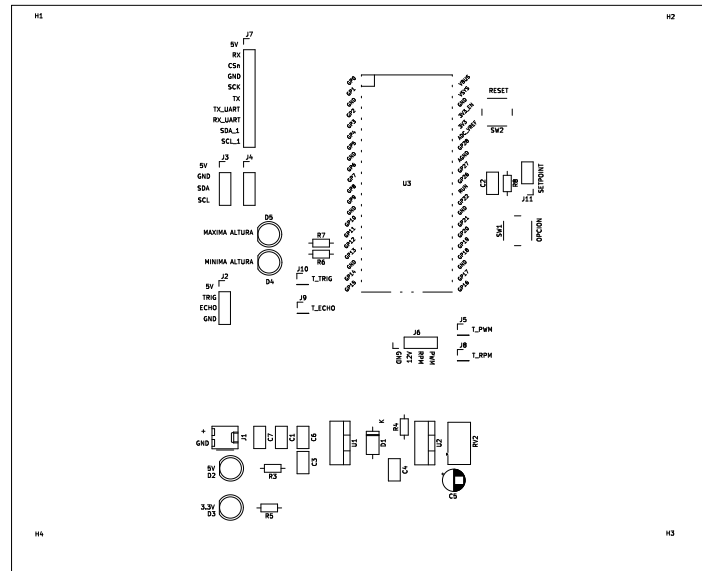
ANEXO 3B: VISTA 3D DEL PCB (LADO DEL COBRE)



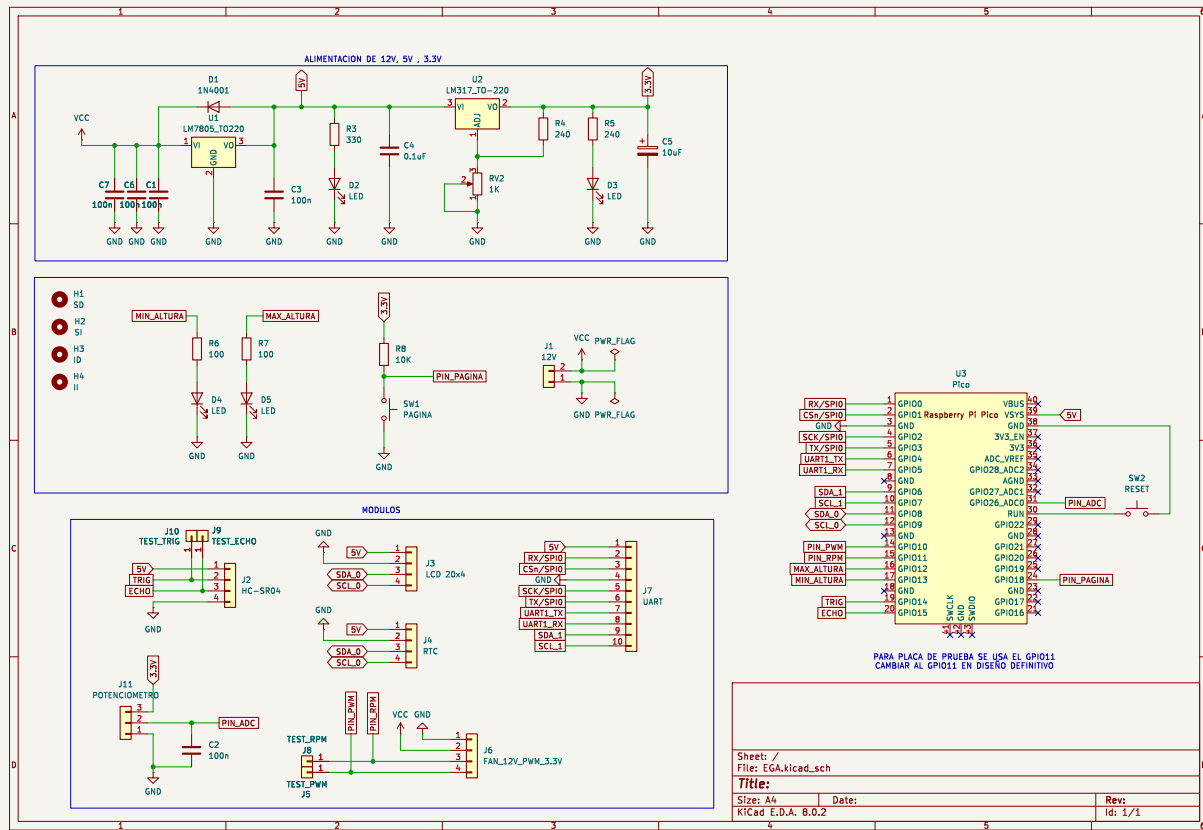
ANEXO 4A: PCB PARA IMPRESO (LADO DEL COBRE) EN ESCALA 1:1



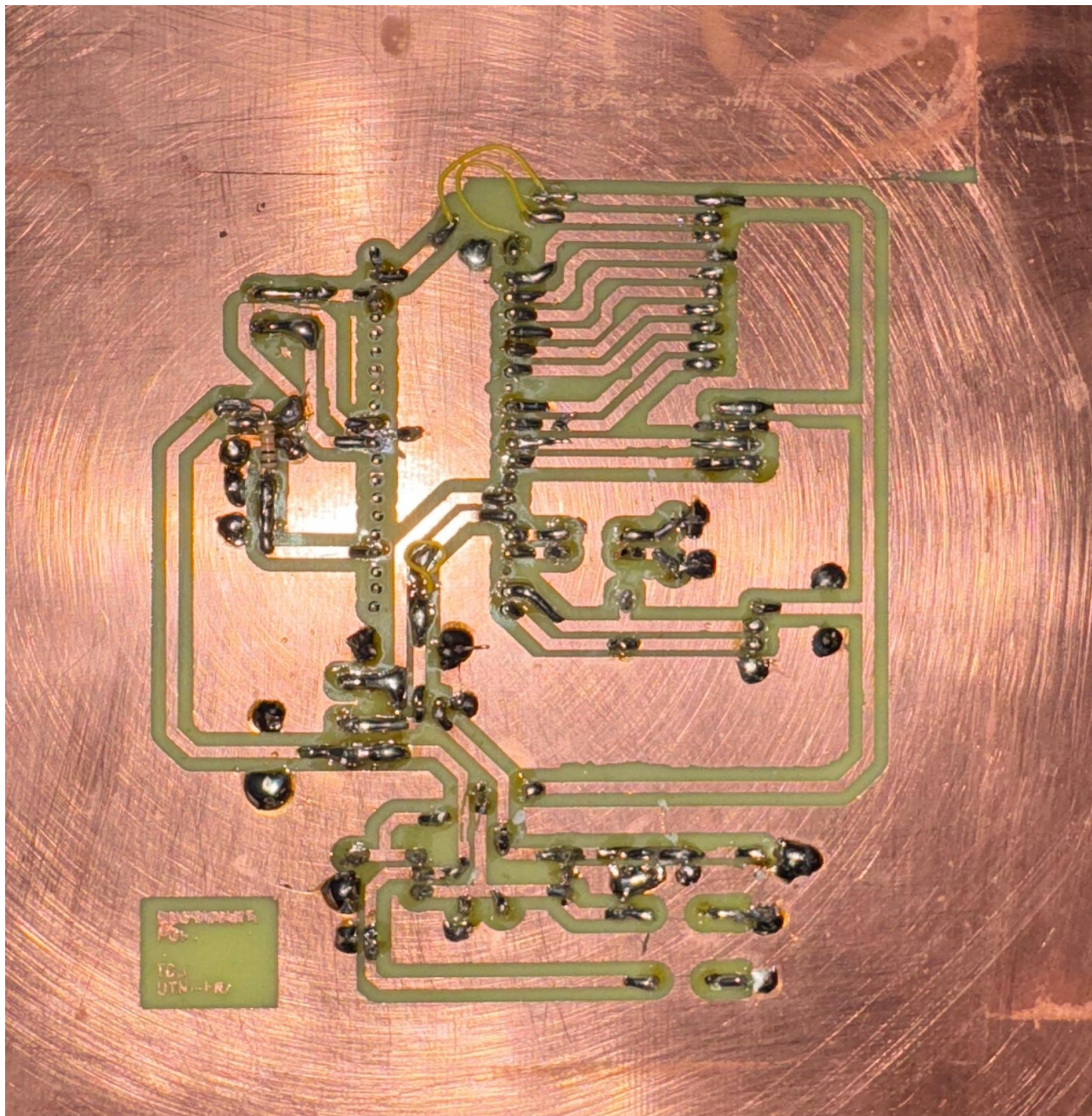
ANEXO 4B: PCB PARA IMPRESO (LADO DE COMPONENTES) EN ESCALA 1:1



ANEXO 5: ESQUEMÁTICO



ANEXO 6: PLACA IMPRESA (LADO DEL COBRE)



ANEXO 6B: PLACA IMPRESA (LADO DE LOS COMPONENTES)

